LifeLog CI/CD: viva study guide

Everything about this project in one place: what it is, how each piece works, why each choice was made, what went wrong and how it was fixed, and 50 likely questions with answers. Read sections 1 to 4 first, then drill section 12.

1. The project in 60 seconds

LifeLog is a small web app for tracking daily life with one text box: the *capture bar*. You type `water 3` or `spent 250 lunch` and it logs it. The app is the vehicle; the assignment is the **CI/CD pipeline** around it.

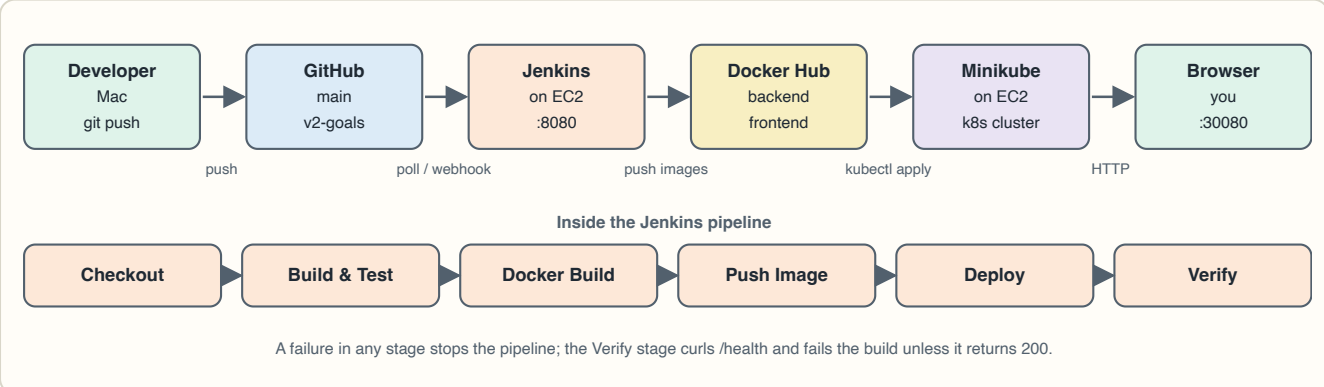
The pipeline: a developer pushes to **GitHub**. **Jenkins** (on an **AWS EC2** server) checks out the code, runs **pytest**, builds two **Docker** images (backend and frontend), pushes both to **Docker Hub**, deploys them to a **Minikube** Kubernetes cluster, and verifies the app answers `/health` with HTTP 200. A second run (merging the `v2-goals` branch) shows a code change reaching the cluster automatically.

Say this if you blank: "Push to GitHub, Jenkins tests it, builds two images, pushes them to Docker Hub, deploys to Kubernetes with a rolling update, and verifies the health endpoint. The footer shows which build and pod I am talking to."

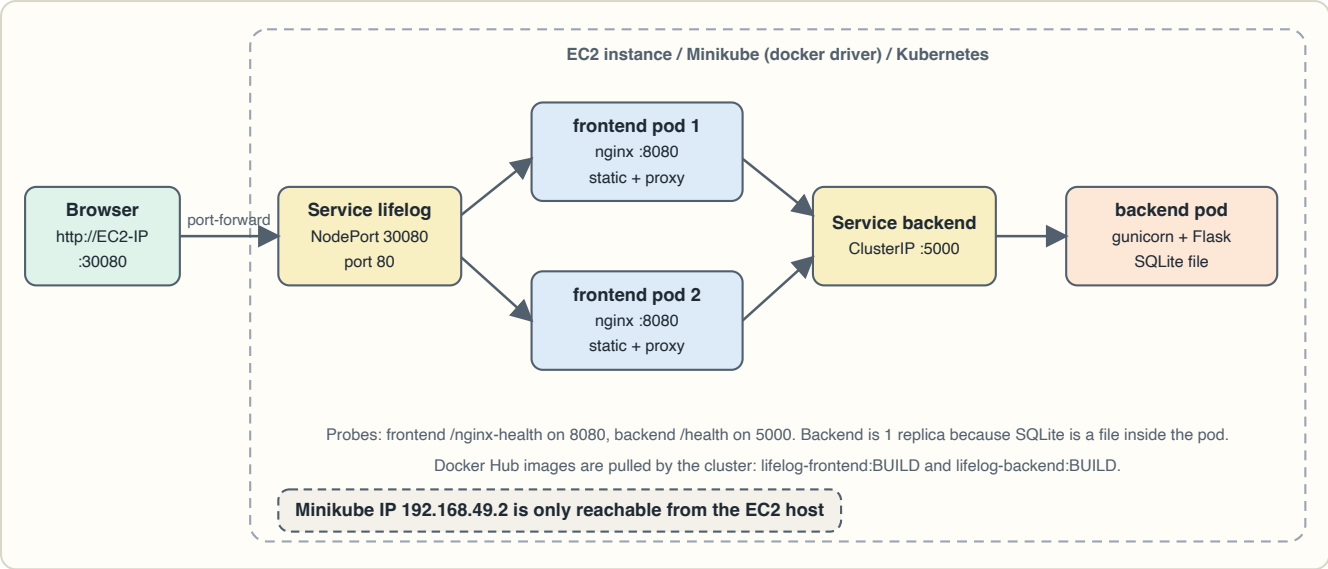
Numbers to remember

Fact	Value
Stack	Python 3.12, Flask, SQLite, gunicorn, nginx
Tests	65 on `main`, 74 on `v2-goals` (pytest)
Images	`saksham122004/lifelog-backend`, `saksham122004/lifelog-frontend`
Tags	build number and `latest`
Server	EC2 (Mumbai, ap-south-1), Ubuntu 24.04, 2 vCPU, 8 GiB, 30 GB disk
Jenkins	port 8080, Java 21, job `lifelog-pipeline`, credential `dockerhub-creds`
Kubernetes	Minikube, docker driver; 2 frontend pods, 1 backend pod
Ports	NodePort 30080 to frontend 8080 to backend 5000
Poll trigger	every 5 minutes (`H/5 * * * *`)

2. Architecture



Reading the diagram. Left to right is the life of a change. Jenkins and Minikube are on the same EC2 machine. Docker Hub sits in the middle as the hand-off point: Jenkins pushes, the cluster pulls.



Runtime path of one request. Browser to port 30080 (a port-forward on the EC2 host) to the `lifelog` Service (NodePort) to one of two nginx pods to the `backend` Service (ClusterIP, port 5000) to the single Flask pod, which reads and writes SQLite.

Component	Job	Why it exists
GitHub	Stores code and branches	Source of truth; the pipeline starts from it
Jenkins	Automation server running the Jenkinsfile	Makes build, test and deploy repeatable
Docker	Packages the app with its dependencies	The same image runs on my Mac, in Jenkins and in the cluster
Docker Hub	Image registry	Jenkins pushes, Kubernetes pulls; tags give version history
Minikube	Single-node Kubernetes cluster	A real Kubernetes API without paying for a managed cluster
nginx (frontend)	Serves static files, proxies the rest	Separates the web tier from the app tier
Flask + gunicorn (backend)	Application logic and the API	gunicorn is a production server; Flask's own server is for development only
SQLite	Database	Simple, file-based, no extra service to run

3. The application

The concept: a one-line log

The main way to use the app is the **capture bar**, pinned to every page. Everything else is a view of what was captured. Press `/` anywhere to focus it. While you type, a **live preview** shows what will happen. Press Enter to apply it.

Trackers

A tracker is something you log daily. It has a name, a type, a colour (one of 8 muted tones), an optional unit and target, a weekday schedule, and an archived flag.

Type	Meaning	Example	Target means
check	done or not done	Gym	none

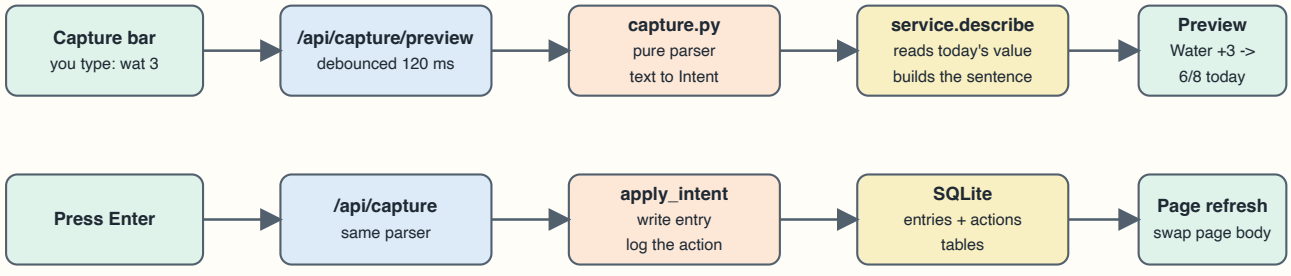
Type	Meaning	Example	Target means
count	reach a daily number	Water	daily target (8 glasses)
number	a value per day	Sleep hours	none
mood	whole number 1 to 5	Mood	none
money	spending with categories	Spending	monthly budget

What the parser understands

`app/capture.py` is a **pure function**: text and the list of trackers in, one *Intent* out. It never touches Flask or the database, which makes it easy to test (30 tests).

You type	Meaning
`gym`, `gym done`, `did gym`	mark the check tracker done today
`water 3`, `water +3`	add 3 to a count tracker
`water =5`	set the count to 5
`sleep 7.5`	set a number tracker
`mood 4`	set mood (rejects 0, 6 and 3.5)
`spent 250 lunch`, `250 food`, `-250 uber`, `₹80 chai`	money entry; category inferred from the keyword, else Other
`yesterday gym`, `gym yesterday`, `mon gym`	backfill; a weekday means the most recent one *before* today
`undo`	reverse the last captured action

Name matching, in order: exact (case-insensitive), then a unique prefix (`wat` finds Water), then a fuzzy match for typos (`watr`). If two names share the prefix the bar shows suggestions (*ambiguous*). If nothing matches it says so and offers a one-click **Create tracker** (*unknown*). A name like "No sugar" is kept intact, because only trailing words such as `done` or `no` are treated as modifiers.



'undo' reads the newest un-undone row in the actions table and reverses it (restores the old value, deletes the money entry, or removes the tracker).

Data model (SQLite)

Table	Columns that matter	Notes
`trackers`	name, type, color, unit, target, days, archived	`days` is a string of weekday digits, Monday = 0
`entries`	tracker_id, day, value, category, note	One row per tracker per day, except money (many rows per day)
`actions`	kind, tracker_id, day, prev_value, entry_id, label, undone	The undo log: what the capture bar did

Smart parts worth mentioning

- **Schedule-aware streaks** (`app/stats.py`): unscheduled days never break a streak, and a scheduled *today* that is not done yet does not break it either.
- **Undo** restores the previous value, deletes a money entry, or removes (or archives) a tracker that the bar created.
- **Progressive enhancement:** the inline buttons are normal HTML forms that work without JavaScript. `app.js` (about 90 lines, no libraries) upgrades them to background requests that swap in fresh page content.
- **Seed data** is generated relative to today when a fresh database is empty, so every new pod opens with a lived-in log.
- **Footer** reads `APP_VERSION`, `BUILD_NUMBER`, `GIT_COMMIT` and `HOSTNAME` from environment variables (default `dev`), plus an `X-Frontend-Pod` header added by nginx.

Code map

File	Role
`app/capture.py`	pure parser
`app/stats.py`	pure streak, completion and sparkline maths
`app/service.py`	all database operations; applies intents; undo
`app/views.py`, `trackers.py`, `insights.py`	HTML pages
`app/api.py`	JSON endpoints for the capture bar
`app/db.py`, `seed.py`	schema, connection, demo data
`app/static/style.css`, `app.js`	the design and the capture bar

4. Docker

Image vs container

An **image** is a read-only recipe plus files (like a class). A **container** is a running instance of it (like an object). Images are built from a **Dockerfile**, layer by layer; unchanged layers are cached, which is why `requirements.txt` is copied and installed *before* the app code.

Backend Dockerfile, line by line

Instruction	What it does
`FROM python:3.12-slim`	small Debian-based Python image
`ARG APP_VERSION, GIT_COMMIT, BUILD_NUMBER`	values passed at build time (`--build-arg`)
`ENV APP_VERSION=${APP_VERSION} ...`	turns build args into runtime environment variables; the footer reads them
`COPY requirements.txt` then `RUN pip install`	dependency layer, cached unless requirements change
`COPY app ./app` and `wsgi.py`	only the code the app needs
`RUN useradd ... USER lifelog`	runs as a **non-root** user (uid 10001); only `/app/data` is writable
`HEALTHCHECK`	Docker marks the container healthy only if `/health` returns 200
`CMD gunicorn --workers 1 --threads 4 wsgi:app`	production server on port 5000

Why one gunicorn worker? SQLite is seeded at startup and is a single file; several processes would race on the first seed. Four threads give concurrency.

Frontend image

`nginxinc/nginx-unprivileged` (runs as non-root on port 8080). It copies `app/static` into `/usr/share/nginx/html/static` and a small `nginx.conf` that: answers `/nginx-health` itself, serves `/static/`, and proxies everything else to `http://backend:5000`, adding `X-Frontend-Pod`.

`.dockerignore`

Keeps the build context small and the image clean: `.git`, `.venv`, tests, docs, k8s, caches and local databases are not sent to Docker.

Docker Compose

`docker-compose.yml` starts both containers on one network so `backend` resolves by name. It is for local testing at `http://localhost:8080`; in production Kubernetes does this job.

Docker HEALTHCHECK vs Kubernetes probes

They are independent. Docker's HEALTHCHECK shows in `docker ps`. Kubernetes **ignores** it and uses its own readiness and liveness probes defined in the manifest.

5. Kubernetes

Objects used

Object	In this project
Pod	smallest unit: one container (backend or nginx)
Deployment	keeps N pods running and performs rolling updates (`lifelog-backend` x1, `lifelog-frontend` x2)
ReplicaSet	created by a Deployment; one per version; this is how a rollback is possible
Service ClusterIP `backend`	stable internal name and virtual IP for the backend pods
Service NodePort `lifelog`	opens port 30080 on the node, forwards to the frontend pods on 8080

Manifest features and why

Feature	Value	Reason
`replicas`	frontend 2, backend 1	frontend is stateless; backend holds a SQLite file, so two copies would have different data
`imagePullPolicy: IfNotPresent`		reuse a cached image; the tag changes every build so this is safe
`readinessProbe`	`/health` (backend), `/nginx-health` (frontend)	a pod gets traffic only when ready; a rolling update waits for it
`livenessProbe`	same paths	restarts a container that hangs
`resources`	e.g. backend request 50m CPU/64Mi, limit 250m/128Mi	scheduler placement and protection from runaway use
Downward API	`HOSTNAME` from `metadata.name`	the pod learns its own name; the footer shows it

Rolling update

The default strategy replaces pods gradually: with `maxSurge` 25 percent and `maxUnavailable` 25 percent, a new pod is started and must pass its **readiness probe** before an old one is removed. For one backend replica, Kubernetes starts a new pod, waits until it is ready, then removes the old one, so there is no downtime. `kubectl rollout status` waits for this to finish.

How the image tag is pinned

The manifests in git say `:latest`. At deploy time Jenkins runs `sed` to change the `image:` line in **its own workspace copy** to `:<BUILD_NUMBER>`, then `kubectl apply`. Result: every deployment is tied to one build, and a rollback is as simple as applying an older tag. (The committed file is never modified.)

The NodePort and Minikube networking problem

Minikube with the **docker driver** runs the "node" as a container with its own IP (`192.168.49.2`) on a Docker network. That IP is reachable from the EC2 host but not from the internet, so `<EC2-public-IP>:30080` does not work even though the security group allows it. Solutions: - The pipeline's Verify stage runs *on* the host and uses `minikube ip`. - For a browser: `kubectl port-forward --address 0.0.0.0 svc/lifelog 30080:80` makes the host itself listen on 30080. - A port-forward attaches to **one specific pod**, so after a rollout it must be restarted. - Production fix: an Ingress controller or a cloud load balancer.

6. Jenkins

What Jenkins is

An open-source **automation server**. A **pipeline** is a script (the `Jenkinsfile`) stored in the repo, so the build process is versioned with the code ("pipeline as code").

Setup decisions

Decision	Reason
Java 21	current Jenkins refuses to start on Java 17
`jenkins` user in the `docker` group	lets pipeline steps run `docker build` without sudo
Minikube started **as** `jenkins`	its kubeconfig lives in `/var/lib/jenkins`; the pipeline's `kubectl` needs it
Plugins: Docker Pipeline, Pipeline Stage View	pipeline Docker steps and the stage-by-stage view
Credential `dockerhub-creds` (username + token)	no secret in the repo or the log

Jenkinsfile walkthrough

Part	What it does
`agent any`	run on any available executor (the same machine)
`environment`	`DOCKERHUB_USER`, image names, `APP_VERSION`
`triggers { pollSCM('H/5 * * * *') }`	checks GitHub every 5 minutes (fallback to a webhook)
`options`	timestamps in logs; no concurrent builds
Checkout	`checkout scm`; saves the short commit hash
Build & Test	creates a venv, installs requirements, runs `pytest -q`

Part	What it does
Docker Build	builds backend and frontend with build args; tags `:BUILD_NUMBER` and `:latest`
Push Image	`withCredentials` + `docker login --password-stdin`, then pushes both images
Deploy to Minikube	`sed` the tags, `kubectl apply -f k8s/`, `kubectl rollout status` for both deployments
Verify	`kubectl get pods,deploy,svc`, then `curl http://$(minikube ip):30080/health` with retries; the build fails unless HTTP 200
`post always`	`docker logout`
`post failure`	prints `kubectl describe` and logs of both deployments

Secrets handling

The credential is injected into environment variables for that block only. The shell uses **single-quoted** `sh '''...'''` so Groovy does not interpolate the secret into the command text, and `--password-stdin` keeps it off the process list. Jenkins also masks the value if it ever appears in output.

Poll trigger vs webhook

A **webhook** is GitHub calling Jenkins instantly on push. It needs GitHub to reach port 8080, which my security group blocks (only my IP is allowed). So the Jenkinsfile has `pollSCM` as a fallback: Jenkins asks GitHub every 5 minutes whether anything changed. For demos I click **Build Now**.

7. AWS

Item	Detail
EC2	a virtual server; here `m7i-flex.large`, 2 vCPU, 8 GiB, free-plan eligible
AMI	Ubuntu Server 24.04 LTS
Key pair	`.pem` private key for SSH; `chmod 400` is required
Security group	a virtual firewall: 22 (SSH), 8080 (Jenkins), 30080 (app), each only from my IP
Storage	30 GiB gp3
Cost control	stop the instance when not in use (a stopped instance keeps its disk but loses its public IP); terminate and delete the volume after submission

8. What happens on every push (the timeline)

1. `git push origin main` sends commits to GitHub.
2. Jenkins notices (poll within 5 minutes, or **Build Now**).
3. It checks out the commit and runs the tests; a failure stops here.
4. It builds the two images with the build number, commit and version baked in.
5. It pushes both images to Docker Hub.
6. It rewrites the image tags, applies the manifests, and waits for the rollout.
7. New pods start, pass readiness, old pods terminate (rolling update).
8. Verify curls `/health` and checks for 200; the build is green.
9. The browser footer now shows the new build number, commit and pod names.

9. The second run (`v2-goals`)

`main` is version 1: Today, Log, Insights, Trackers. The branch `v2-goals` adds a **Goals** module (name, target number, current progress, deadline, percent complete) with its own tests. Merging it into `main` and pushing triggers a new build. After it is green, the nav has a Goals link, the footer shows the new build and version 2.0.0, and the pod names are new. This proves the pipeline needs **no manual deployment**.

10. Problems we hit and how they were solved

These make strong viva answers because they show real understanding.

Problem	Cause	Fix
Jenkins apt repo said `NO_PUBKEY`	Jenkins rotated its package signing key	installed the new key and verified its fingerprint with `gpg`
Jenkins would not start	current Jenkins requires Java 21, not 17	installed OpenJDK 21 and made it the default
Wanted `t3.medium` but account is on the Free Plan	that type is not eligible, and I did not want to pay	used a free-plan eligible 8 GiB type (`m7i-flex.large`)
App not reachable at `<EC2-IP>:30080`	Minikube docker driver network is internal	`kubectl port-forward --address 0.0.0.0`
Site stopped after a deploy	a port-forward is bound to one pod, which was replaced	restart the port-forward after rollouts
Data differed between refreshes	two app pods each had their own SQLite file	split into frontend (2 replicas) and backend (1 replica)
`git` commands failed on the server	repo is on my Mac, not the EC2 instance	run git on the Mac; Jenkins has its own checkout
GitHub webhook shows a red cross	port 8080 is closed to GitHub	`pollSCM` fallback every 5 minutes
SSH dropped mid-session	idle timeout	`-o ServerAliveInterval=30`

11. Limitations and improvements

Limitation	Improvement
Data lives in the pod and is lost when it is replaced	PersistentVolumeClaim, or an external database such as PostgreSQL
Backend cannot scale beyond one replica	move to PostgreSQL, then scale and add a PodDisruptionBudget
Port-forward is a manual step	Ingress controller or a cloud load balancer
Webhook cannot reach Jenkins	restrict by GitHub's IP ranges, or use a reverse proxy with HTTPS
Single environment	add staging and production, with a manual approval step
No image scanning	add Trivy or similar to the pipeline
Tests run on the host's Python	run tests inside the same Docker image
Manual `kubectl apply` of raw YAML	Helm or Kustomize; GitOps with Argo CD
No monitoring	Prometheus and Grafana, log aggregation

12. Likely viva questions

CI/CD concepts

1. What is CI/CD? CI (continuous integration) automatically builds and tests every change. CD (continuous delivery or deployment) automatically releases the tested build to an environment. Here: push, test, build images, push to the registry, deploy to Kubernetes.

2. Is this continuous delivery or continuous deployment? Continuous deployment to one environment: every green build on `main` goes straight to the cluster with no manual approval. In a real team I would add staging and an approval gate.

3. Why use a pipeline instead of building by hand? It is repeatable, fast, documented in the repo, catches failures early with tests, and removes "works on my machine".

4. What is "pipeline as code"? The Jenkinsfile lives in the repo. The build process is reviewed, versioned and rolled back with the code.

5. Declarative vs scripted pipeline? Declarative has a fixed structure (`pipeline { stages { ... } }`), is easier to read and validate. Scripted is free-form Groovy. I used declarative.

6. Webhook vs polling? A webhook pushes an event from GitHub to Jenkins immediately. Polling asks GitHub every N minutes. My security group blocks GitHub from port 8080, so I use `pollSCM` as the fallback.

7. What stops a broken change from being deployed? The Build & Test stage runs pytest first. Any failing test fails the build before an image is created. The Verify stage is a second gate after deployment.

Docker

8. Image vs container? An image is the packaged, read-only template; a container is a running instance of it.

9. Why two images? Separation of concerns: nginx serves static files and proxies; Flask handles logic. They scale and update independently (frontend 2 replicas, backend 1), and each image is small and single-purpose.

10. Why `python:3.12-slim`? Small and quick to pull, but still a full Debian userland so pip packages install without extra tools.

11. Why run as non-root? If the app is compromised the attacker has no root privileges inside the container. Only `/app/data` is writable.

12. Why copy `requirements.txt` before the code? Docker caches layers. Dependencies change rarely, so that layer is reused and builds are fast when only code changes.

13. What are build args? Values passed at build time with `--build-arg`. I turn them into `ENV` so the running app can show version, commit and build number.

14. Why `--workers 1 --threads 4` for gunicorn? SQLite is one file and the app seeds it at startup; multiple worker processes could race. Threads give concurrency inside one process.

15. What does HEALTHCHECK do? Docker runs the command periodically and marks the container healthy or unhealthy. Kubernetes does not use it; it uses probes.

16. Why tag with the build number and `latest`? The build tag identifies exactly what was deployed and allows rollbacks. `latest` is a convenience pointer.

17. What is `.dockerignore` for? It excludes files (git history, virtualenv, tests, docs) from the build context: smaller, faster, safer builds.

18. Why nginx in front of Flask? nginx serves static files efficiently, can terminate TLS, and isolates the app server. Here it also adds the frontend pod header.

Kubernetes

19. What is a Pod? A Deployment? A ReplicaSet? A pod runs one or more containers. A Deployment declares the desired state and manages ReplicaSets. A ReplicaSet keeps a set number of identical pods; each version of a Deployment gets its own ReplicaSet.

20. Service types? ClusterIP (internal only, used for `backend`), NodePort (a port on every node, used for `lifelog`), LoadBalancer (cloud load balancer, not used here).

21. Readiness vs liveness probes? Readiness decides if a pod receives traffic and gates rolling updates. Liveness decides if a container must be restarted.

22. How does a rolling update work? New pods start, must become ready, and only then are old pods removed, so there is no downtime. Old ReplicaSets remain for rollback.

23. What is the downward API? It exposes pod metadata to the container, here the pod's own name through the `HOSTNAME` environment variable. The footer shows it.

24. Why requests and limits? Requests tell the scheduler how much to reserve; limits cap usage so one pod cannot starve the others.

25. Why `imagePullPolicy: IfNotPresent`? It reuses a cached image. This is safe because every deployment uses a unique build-number tag.

26. How do you roll back? `kubectl rollout undo deployment/lifelog-backend`, or re-apply with an older image tag, which Docker Hub keeps.

27. How does the frontend find the backend? Kubernetes DNS: the Service named `backend` resolves inside the cluster, so nginx proxies to `http://backend:5000`.

28. Why is the backend one replica? It stores data in a SQLite file inside the pod. Two replicas would each hold different data. A shared database would allow scaling.

29. What is Minikube? A tool that runs a single-node Kubernetes cluster locally or on one server, ideal for learning and demos.

30. Why can't I open `<EC2-IP>:30080` directly? With the docker driver the Minikube node has an internal IP (`192.168.49.2`) on a Docker network. I forward the service on the host with `kubectl port-forward --address 0.0.0.0`.

Jenkins

31. Why is the `jenkins` user in the `docker` group? Pipeline steps run as `jenkins`; without group membership they would get "permission denied" on the Docker socket.

32. Why start Minikube as the `jenkins` user? The kubeconfig and cluster belong to the user who started it. The pipeline's `kubectl` runs as `jenkins`, so it needs access to that cluster.

33. How are secrets handled? A Jenkins credential with a Docker Hub access token, injected with `withCredentials`, passed with `--password-stdin`, and masked in logs. Nothing is in the repo.

34. Why a token and not my password? A token can be scoped (read and write only), has an expiry, and can be revoked without changing my password. If Jenkins is compromised, the account password is not exposed.

35. What does the Verify stage do and why? It checks that the deployed app really answers: `curl /health` through the NodePort and fail on anything but 200. Deploy succeeding does not prove the app

works.

36. What does `post always` and `post failure` do? `always`: log out of Docker Hub. `failure`: print `kubectl describe` and logs so the cause is visible in the Jenkins console.

37. What if two pushes happen at once? `disableConcurrentBuilds()` queues them, so deployments do not overlap.

AWS and security

38. What is a security group? A stateful virtual firewall for the instance. Mine allows 22, 8080 and 30080 only from my IP.

39. Why did you pick this instance type? Jenkins plus Minikube need about 4 GB, and my account was on the Free Plan, which blocks `t3.medium`. I used a free-plan eligible 8 GiB type.

40. How do you control cost? Stop the instance when not in use, set a budget alert, and delete the instance and volume after submission.

41. What would you secure further? HTTPS, restrict Jenkins to a VPN or reverse proxy, scan images, use a secrets manager, and rotate the Docker Hub token.

The application

42. How does the capture parser work? It splits the text into tokens, removes day words (`yesterday`, `mon`), pulls out the first number, strips trailing words like `done`, then finds the longest leading phrase that names a tracker (exact, prefix, then fuzzy). The tracker's type decides what to do with the number. The result is an Intent object.

43. How is undo implemented? Every applied capture writes a row in the `actions` table with the previous value or the created entry id. `undo` reverses the newest row that is not undone.

44. Why is the parser a pure function? No Flask or database dependency means it can be unit-tested with plain data, which is why it has 30 fast tests.

45. How are streaks computed? Walking back from today over scheduled days only. A missed scheduled day ends the streak, but today not being done yet does not.

46. How does the footer show the build and pod? The Dockerfile turns build args into environment variables; Jenkins passes the build number and commit. The pod name comes from the downward API, and nginx adds the frontend pod name as a header.

Scenarios

47. A bug reaches production. What do you do? Roll back with `kubectl rollout undo` (or redeploy the previous tag), then fix forward with a new commit and a regression test.

48. The pipeline passes tests but the app is down. How is that possible and caught? Tests do not cover cluster configuration. The rollout wait and the Verify stage catch it, and the failure block prints pod logs.

49. How would you add a staging environment? A second namespace or cluster, a branch or tag rule in the Jenkinsfile, deploy to staging first, run smoke tests, then require a manual approval before production.

50. How would you make the data persistent? Add a PersistentVolumeClaim and mount it at `/app/data`, or move to PostgreSQL. The backend would then be able to scale.

13. Command cheat sheet

On the Mac

```
cd ~/lifelog-cicd
.venv/bin/pytest -q                       # run tests
docker compose up -d --build              # run both images locally, http://localhost:8080
docker images | grep lifelog
docker compose down
git checkout main && git pull
git merge --no-ff v2-goals -m "Merge v2-goals: add Goals module" && git push origin main
ssh -i ~/Downloads/lifelog-key.pem -o ServerAliveInterval=30 ubuntu@<EC2-IP>
```

On the EC2 instance

```
sudo -u jenkins -H minikube status
sudo -u jenkins -H minikube start --driver=docker --cpus=2 --memory=4096
sudo -u jenkins -H kubectl get pods,deploy,svc
sudo -u jenkins -H kubectl get pods -w
sudo -u jenkins -H kubectl logs deploy/lifelog-backend --tail=50
sudo -u jenkins -H kubectl describe deploy lifelog-backend
sudo -u jenkins -H kubectl rollout history deployment/lifelog-backend
sudo -u jenkins -H kubectl rollout undo deployment/lifelog-backend
sudo -u jenkins -H minikube ip
sudo systemctl status jenkins
sudo journalctl -u jenkins --no-pager | tail -30
sudo pkill -f "kubectl port-forward"
sudo -u jenkins -H nohup kubectl port-forward --address 0.0.0.0 svc/lifelog 30080:80 >
/tmp/portforward.log 2>&1 &
curl -s localhost:30080/health
```

14. Glossary

Term	Meaning
CI / CD	continuous integration (build and test every change) / continuous delivery or deployment
Pipeline	an automated sequence of stages, defined in the Jenkinsfile
Stage	one named step of the pipeline
Registry	a store for container images (Docker Hub)
Tag	a label on an image, here the build number or `latest`
Pod	the smallest deployable unit in Kubernetes
Deployment	a controller that keeps pods running and updates them
Service	a stable network address in front of pods
NodePort	a Service type that opens a port on the node
Probe	a health check Kubernetes runs on a container
Rolling update	replacing pods gradually so the app stays up
Port-forward	tunnels a local port to a Service or pod
Downward API	gives a pod information about itself
Minikube	a single-node Kubernetes cluster for learning
EC2	AWS virtual server
Security group	AWS firewall for an instance

Term	Meaning
pollSCM	Jenkins checking the repository on a schedule
Webhook	an HTTP callback GitHub sends when something happens
Intent	the parsed result of a capture-bar line (what to do, to which tracker, which day)